University of Luxembourg

Introduction to Machine Learning

Assignment 2

Total marks:	20 points
Due date:	25 November 2026, 23:59
Submission:	Moodle
Resources:	https://github.com/cgadski/iml_2026.git
Responsible:	Haya Ejjawi (`haya.ejjawi@uni.lu`)
	Arthicha Srisuchinnawong (`arthicha.srisuchinnawong@uni.lu`)

General Instructions

1. Before starting, please check the Large Language Models (LLM) usage policy and be aware that none of the exercises here require the use of an LLM. (`2026-Guidelines-AI-I2TL-16022026.pdf`)

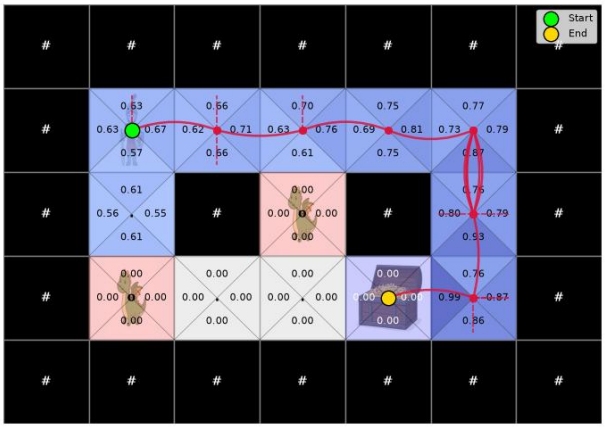
2. Complete both parts of the assignment (20 points in total). Show your reasoning and intermediate steps for mathematical questions. It is recommended to address the different questions sequentially as each question providing context for the next.

3. Present mathematical derivations and explanations in Markdown cells using LaTeX notation. Use code cells for implementations and experiments. Write readable, well-structured, and appropriately documented code.

4. Ensure that your notebook runs sequentially from a fresh kernel using the course environment. Document any additional dependencies and use random seeds for reproducibility where appropriate.

5. Include all requested figures with appropriate labels and interpretations. Submit both the completed Jupyter notebook (`.ipynb`) and its HTML export (`.html`), with your name(s) included, following the course repository instructions. The Jupyter notebook should be run-able given the module listed in `uv.lock` provided. The code should be straightforward and readable; just make it clear enough to debug or grasp the general idea quickly.

Part 1: An RL knight in a gridworld (10 points)



Preamble

Congratulations! You have been knighted. Your first mission as a knight is to navigate various dungeons and bring back treasure to help our kingdom prosper. The aim of this first mission (and this problem) is to help you develop an intuition for the different parameters of a simple reinforcement learning agent, particularly a Q-learning agent. You will also experience a problem called reward hacking through trial and error, which is a (hopefully) fun challenge in reinforcement learning.

The `Dungeon` class in `iml/dungeon.py` defines a simple gridworld environment, created by using maps generated in `iml/dungeon_maker.py`. The `Agent` class in `iml/agent.py` defines the Q-learning agent (the knight) to be programmed externally (see coding template), and the `DungeonVisualization` class in `iml/visualization.py` provides the necessary plots to observe and interpret the agent's strategies.

States represent positions within a dungeon. Some states are unreachable because they are occupied by walls (#) or dragons (o). When you try to walk into a wall, your position remains unchanged. When you try to walk into a dragon, you are unalived and respawned at the starting position. You can take four possible integer actions $\{0, 1, 2, 3\}$, each representing a direction in which the agent chooses to move. Your goal is to navigate from the starting position `start` (s) to the goal position `end` (e, representing the treasure) as 'optimally' as possible.

Remember that a policy is a rule specifying which actions the agent should take as a (potentially stochastic) function of its current state. For a given policy $\pi(a|s)$, or simply π, let $V_\pi(i)$ be the expected return that an agent following the policy will receive per episode (running for a specific number of steps).

1.1 A drunken knight in a treasure room (4 points)

use default `dungeon = make_treasure_room()` with 0 % slippery probability.

(a) (0.5 points) Implement a stochastic policy π where the you select actions uniformly at random.
Hint: You may use `agent.run_episode(dungeon, nstep, gamma = 1)` to run the agent and retrieve the state, action, and reward trajectory.

[provide your code here]

(b) (0.5 points) Observe how you move and draw a Markov decision process (MDP) diagram for all four non-wall states.
Hint: Lecture 8, Slide 35 (40/48).

[draw, capture, and insert your photo/figure here]

(c) (1 point) Compute the theoretical probability distribution over states at time step $t = 2$ numerically (without writing code) when you use the stochastic policy you implemented.
Hint: In conditional probability and Markov processes, the probability distribution after N steps is obtained by marginalizing over all intermediate state trajectories:

$$P(s^{(N)} = s_N \mid s^{(0)} = s_0) = \sum_{s_1,\ldots,s_{N-1}} \prod_{t=1}^{N} P(s^{(t)} = s_t \mid s^{(t-1)} = s_{t-1})$$

where $s^{(t)}$ denotes the state at time step t. You can compute the N-step distribution by taking the product of single-step transition probabilities and summing over all possible intermediate states $s_1, \ldots, s_{N-1}$.

(d) (1 point) If you implement Q-learning with an episode length of 1 (`nstep = 1`) under current setup, will you manage to perform better? Why or why not? Explain your reasoning.
Hint: You may refer to your answer from the previous question (1.c) and are encouraged to run your program multiple times for testing to verify your answer.

(e) (1 point) For a dungeon of width N and height M, what is the minimum number of steps required to reach the goal from the starting position? Why?
Hint: You may refer to your answer from the previous questions (1.c/d) and are encouraged to run your program multiple times for testing to verify your answer.

1.2 A tricky knight in an underground maze (6 points)

use default `dungeon = make_underground_maze()` with 0 % slippery probability.

(a) (1 point) Using your stochastic policy π, implement the Q-learning update with a discount factor ($\gamma = 1$) and a maximum episode length of 30 steps ($n_{\text{step}} = 30$). Run an initial episode, followed by several consecutive episodes without resetting the agent's Q-table. Explain your observations on how the Q-values change overtime relative to the goal and starting locations.

Hint: You may consider the evolution of the Q-table and success rate.

[Insert ≈ 3 selected snapshots (without resetting the agent) as your empirical proof here]

(b) (1 point) By varying the discount factor γ (e.g., trying values such as 1.0, 0.99, 0.9, $\cdots$, 0.0), do you observe any differences? Summarize how the discount rate affects the results and explain why it is necessary.

Hint: What happens to the agent's behavior at low (≈ 0.1), high (≈ 1.0), and intermediate γ values? You may also use `<vis>.show(states, actions, <knight>.q_values)` to visualize your path, use `<vis>.plot_training_reward()` to monitor your performance, and/or create a customized plot to support your observation. Remember to update the `gamma` parameter in `<agent>.run_episode(gamma=1.0)` with your specific γ value used in Q-learning as well.

[Insert at least 2 selected snapshots (and perhaps your customized code) as your empirical proof here]

(c) (1 point) Implement an ϵ-greedy policy for your agent by varying ϵ (e.g., trying values such as 0.0, 0.2, 0.5, 0.8, 0.99, and 1.0). Plot the sum of discounted rewards per episode over 100 training runs and explain your observations.

Hint: You may use `<vis>.show(states, actions, <knight>.q_values)` to visualize your walking strategy and `<vis>.plot_training_reward()` to monitor your performance.

[Insert your code and at least 3 selected snapshots as empirical proof here]

(d) (0.5 points) Is a single training run sufficient to evaluate and compare an agent's performance across different hyperparameter settings? Specifically: (i) Will an agent reach the goal on every evaluation episode? Explain why or why not. (ii) Under what specific conditions would RL training and evaluation results be strictly deterministic and reproducible? And (iii) Given these factors, how should an RL agent's performance be properly evaluated and compared, both when those deterministic conditions are met and when they are not?

(e) (1 point) Recreate the maze using `make_underground_maze_with_meat()`, where you get half of the reward when stepping into the meat states (+); re-implemented the reward function; set $\epsilon = 0.9$, $n_{\text{step}} = 50$, and $\gamma = 0.95$, and run the agent for 100 to 200 training runs. Explain your observations regarding your strategy (Q-values) and behavior.

(f) (1.5 points) To understand the previous result, compare the state value $V^\pi(s_0)$ under three deterministic strategies from the start state s_0: (1) going directly to the treasure, (2) collecting free meat as you observed, or (3) attempting both. Explain your results numerically.

Hint: Under a deterministic trajectory $(s_0, a_0, s_1, a_1, \dots)$, the state value under policy π satisfies the Bellman expectation equation:

$$V^\pi(s) = R(s, \pi(s)) + \gamma V^\pi(s')$$

Compute the discounted return $V^\pi(s_0) = \sum_{t=0}^{T} \gamma^t R(s_t, a_t)$ for each strategy to determine which path maximizes the total reward.

(g) (Bonus, +0.5 points) How might you modify the reward function or environment design to prevent this *reward hacking*? Propose and explain a solution.
Hint: you should refer to your previous answer(s).

1.3 A rookie knight in a dragons' den (0 points, optional)

use default `dungeon = make_dragons_den(slipper=0.1)` with 10% slip probability (slip to the right of the intended direction)

You may customize your dungeon (`iml/dungeon_maker.py`), reward, or strategy. Are there any interesting condition or scenario you would like to share?

Part 2: Bayesian Decision-Making in Multi-Armed Bandits (10 points)

Preamble

Consider a multi-armed bandit problem with five arms (slot machines). At each time step t, an agent selects an arm $A_t \in \{0, 1, 2, 3, 4\}$ and receives a binary reward $R_t \in \{0, 1\}$, representing failure or success.

Each arm i has an unknown success probability θ_i. The rewards follow a Bernoulli distribution:

$$R_t \mid A_t = i, \theta_i \sim \text{Bernoulli}(\theta_i)$$

with probability mass function

$$P(R_t = r \mid A_t = i, \theta_i) = \theta_i^r (1 - \theta_i)^{1-r}, \qquad r \in \{0, 1\}.$$

The objective is to maximize cumulative reward while balancing two competing objectives:

- **Exploration:** Selecting arms to gather information about their unknown reward probabilities.
- **Exploitation:** Selecting arms believed to provide the highest expected reward.

Part 1 explores ϵ-greedy action selection, which use ϵ for balancing greedy action selection with random exploration. In Part 2, you will investigate Thompson sampling, a Bayesian approach that uses uncertainty about unknown reward probabilities (i.e., from previous experience) to guide action selection.

Bayesian background

Bayesian inference represents uncertainty about an unknown parameter θ using a probability distribution. Before observing data, we specify a prior distribution $p(\theta)$. After observing data D, we update this distribution using Bayes' theorem:

$$p(\theta \mid D) = \frac{p(D \mid \theta) p(\theta)}{p(D)}$$

where $p(D \mid \theta)$ is the likelihood and $p(\theta \mid D)$ is the posterior distribution.

A convenient distribution for an unknown probability $\theta \in [0, 1]$ is the Beta distribution:

$$\theta \sim \text{Beta}(\alpha, \beta)$$

with probability density

$$p(\theta) = \frac{1}{B(\alpha, \beta)} \theta^{\alpha-1}(1-\theta)^{\beta-1}, \qquad 0 \leq \theta \leq 1.$$

Here, $\alpha, \beta > 0$ are shape parameters, and $B(\alpha, \beta)$ is a normalization constant. The mean of a Beta distribution is

$$\mathbb{E}[\theta] = \frac{\alpha}{\alpha + \beta}.$$

A prior is called **conjugate** to a likelihood if the posterior distribution belongs to the same distribution family as the prior.

2.1 Bayesian Estimation of Arm Rewards (2.5 points)

Consider an arm with an unknown probability of success θ, with prior distribution

$$\theta \sim \text{Beta}(\alpha, \beta).$$

Suppose we observe s successes and f failures from independent pulls of the arm, conditional on θ.

(a) (1 point) Starting from Bayes' theorem and the Bernoulli likelihood, derive the posterior distribution $p(\theta \mid D)$ and identify its updated parameters. Hence, show that the Beta distribution is a conjugate prior to the Bernoulli likelihood.

Hint: Use the conditional independence of the observations to construct the likelihood of the complete dataset.

(b) (1.5 point) Suppose the prior is $\text{Beta}(2, 2)$. After ten pulls, the arm produces seven successes and three failures. Compute the posterior distribution and posterior mean.

Show that, in general, the posterior mean can be expressed as a weighted average of the prior mean and the empirical success rate. Interpret the weights and explain how they change as the number of observations increases (*Hint:* what happens as $s + f \to \infty$?).

2.2 Learning with Thompson Sampling (5 points)

Consider a five-armed Bernoulli bandit with true success probabilities

$$\boldsymbol{\theta} = (0.15,\ 0.35,\ 0.55,\ 0.65,\ 0.80).$$

These probabilities are used by the environment to generate rewards. The learning agent must not use the true probabilities when selecting actions. Assume that each arm initially has an independent $\text{Beta}(1, 1)$ prior.

Let $\alpha_{i,t}$ and $\beta_{i,t}$ denote the posterior parameters of arm i before decision t. Thompson sampling proceeds as follows:

1. For each arm i, independently sample a possible success probability from its current posterior distribution:
$$\tilde{\theta}_{i,t} \sim \text{Beta}(\alpha_{i,t}, \beta_{i,t}).$$

2. Select the arm with the highest sampled value:
$$A_t = \arg\max_i \tilde{\theta}_{i,t}.$$

3. Observe the reward R_t from the selected arm.

4. Update the posterior distribution of the selected arm using the new observation.

(a) (3 points) Implement a five-armed Bernoulli bandit environment that generates binary rewards using the given success probabilities. Then implement a Thompson sampling agent that maintains the posterior parameters for each arm, selects actions using posterior sampling, and updates the corresponding posterior after receiving each reward.

Run Thompson sampling for 2,000 decisions. Ensure that the agent does not access the true reward probabilities when selecting actions.

(b) (2 points) Using the results from part (a), plot the evolution of the posterior mean estimates for all five arms as a function of time. Additionally, visualize the posterior distributions of all five arms at the beginning ($t = 0$), after 200 completed decisions ($t = 200$), and after 2,000 completed decisions ($t = 2000$). Here, t denotes the number of completed decisions for the purpose of these plots, with $t = 0$ representing the initial prior distributions.

Discuss how the posterior means and distributions evolve. Explain why some arms become more certain about their estimated success probabilities than others.

2.3 Experimental Evaluation and Exploration (2.5 points)

We now evaluate how frequently Thompson sampling selects the optimal arm as learning progresses and compare its exploration strategy with ϵ-greedy.

(a) (1.5 points) Perform 100 independent trials of Thompson sampling, each consisting of 2,000 decisions. Initialize a new agent with the same priors at the beginning of every trial.

Let i^* denote the arm with the highest true success probability. At each time step t, estimate the fraction of trials in which the agent selects this arm:
$$\hat{p}_t = \frac{1}{100} \sum_{j=1}^{100} \mathbb{I}(A_t^{(j)} = i^*).$$

Here, $A_t^{(j)}$ denotes the arm selected at decision t in trial j, and $\mathbb{I}(\cdot)$ is the indicator function, equal to 1 when its condition is true and 0 otherwise. Plot $\hat{p}_t$ as a function of t. Describe and interpret the observed trend.

(b) (1 points) Compare ϵ-greedy exploration with Thompson sampling. Explain why Thompson sampling may select an arm with a lower posterior mean and how its exploration behavior changes as more observations accumulate.

(c) (Bonus, +1 point) Consider an ϵ-greedy agent that has already correctly identified an optimal arm. At each decision t, with probability $1 - \epsilon$, the agent follows its greedy policy, which selects an optimal arm. With probability ϵ, it explores by selecting uniformly at random among all K arms.

Let A_t denote the arm selected at decision t, and let $\theta^* = \max_i \theta_i$ denote the optimal expected reward per decision. We define the cumulative pseudo-regret after T decisions as:

$$\mathcal{R}_T = \sum_{t=1}^{T} (\theta^* - \theta_{A_t}),$$

which measures the total expected-reward gap associated with the selected arms.

Derive the agent's expected reward per decision and its expected instantaneous pseudo-regret. Hence, derive the expected cumulative pseudo-regret $\mathbb{E}[\mathcal{R}_T]$. How does it grow with T, assuming ϵ remains fixed and the greedy choice remains optimal?